Lab 4 VibeCoding: Chat History

Assigned repo: SETAPESU26/07_catch_falling · AI tool: Claude (Claude Code) · Session: https://claude.ai/code/session_01FKzraxEq2zxWNqPX5Vv1FX · Exported 23 Oct 2026
Lab 4 portion of the session. Claude's tool calls (shell commands, file edits) are summarised in prose; the code changes are the actual diffs from each commit.

STUDENT

```
@Lab_4_VibeCoding_Student_handout.pdf  Deliverables: Upload the following to your Lab repo
under Lab-4 folder,
a. Videos (before and after)
b. Updated Code
c. Chat history exported as a doc/pdf

Refer worksheet for the code assigned to you
https://github.com/SETAPESU26/07_catch_falling

this is my requirment for my lab submission
so do all requirment and get me final submission output so that i can upload it
```

CLAUDE (CLAUDE CODE)

I read the Lab 4 handout and cloned `SETAPESU26/07_catch_falling`. The README describes one deliberate bug with two symptoms (Task 1) and three features (Tasks 2–4):

- **Task 1:** `is_caught()` only checks x, so an object scores the moment it spawns if the basket is under it; and the catch loop in `update()` removes items from the list it is iterating, which can skip an object.
- **Task 2:** the boundary clamp uses `0..WIDTH` for the basket's centre, so the basket hangs half off either edge.
- **Task 3:** spawning is a fixed 50-frame interval with a plain random x.
- **Task 4:** no speed boost exists (the `Basket` has unused `boosted_frames` / `boost_speed` fields).

Plan: add the untouched starter under `Lab-4/`, record a 10-second "before" video, fix each task in its own commit with a test, record a 10-second "after" video, and export this chat as a PDF.

I installed pygame 2.6.1 and wrote a headless recorder that plays the game with scripted arrow-key input, overlays a caption naming what is being shown, and encodes 10 s of H.264 video with ffmpeg. The first "before" take had the caption bar covering the basket and the right-edge moment came too late, so I moved the caption to the top and retimed the key presses. The final "before" video shows the score rising while objects are still at the top of the screen, and the basket sliding half off the right and then the left edge.

CLAUDE (CLAUDE CODE)

Task 1: Fix the falling-object collision bug

Catches now need the object to be over the basket *and* at basket height (its bottom edge has reached the basket top and it hasn't fallen past). The loop now collects every catch first and then rebuilds the list, so no object is skipped.

```
diff --git a/Lab-4/catch-falling/game/collision.py b/Lab-4/catch-falling/game/collision.py
index d5cd814..c3cf806 100644
--- a/Lab-4/catch-falling/game/collision.py
+++ b/Lab-4/catch-falling/game/collision.py
@@ -5,3 +5,9 @@ collision: figures out whether a falling object is within the basket.

 def is_caught(basket_rect, obj):
-    return basket_rect.left <= obj.x <= basket_rect.right
+    # Horizontal: the object's centre must be over the basket.
+    within_x = basket_rect.left <= obj.x <= basket_rect.right
+    # Vertical: the object's bottom edge must have reached the basket's top
+    # edge, and the object must not already have fallen past the basket.
+    at_basket_height = (obj.y + obj.radius >= basket_rect.top
+                        and obj.y - obj.radius <= basket_rect.bottom)
+    return within_x and at_basket_height
```

```
diff --git a/Lab-4/catch-falling/game/game_engine.py b/Lab-4/catch-falling/game/game_engine.py
index 9fa21ad..b77810a 100644
--- a/Lab-4/catch-falling/game/game_engine.py
+++ b/Lab-4/catch-falling/game/game_engine.py
@@ -62,9 +62,11 @@ class GameEngine:
             obj.update()

+        # Collect every catch first, then rebuild the list, so removing one
+        # object can never make the loop skip the object after it.
         basket_rect = self.basket.get_rect()
-        for obj in self.objects:                # BUG: mutating this list while iterating over it
-            if is_caught(basket_rect, obj):
-                self.score += 1
-                self.objects.remove(obj)
+        caught = [obj for obj in self.objects if is_caught(basket_rect, obj)]
+        if caught:
+            self.score += len(caught)
+            self.objects = [obj for obj in self.objects if obj not in caught]

         missed = [o for o in self.objects if o.is_past_bottom(HEIGHT)]
```

Test: an object at the top is not caught, an object at basket height is caught, and two adjacent catchable objects both score. The same test fails on the original starter (`AssertionError: caught at top`) and passes after the fix.

CLAUDE (CLAUDE CODE)

Task 2: Improve basket control

Movement moved into `Basket.move()`. Velocity eases toward the target speed (fast enough to feel responsive, without jerky starts), held keys give continuous motion, and pressing both arrows cancels out. `clamp()` keeps the basket's centre within `[width/2, WIDTH − width/2]`, so the whole basket stays on screen.

```
diff --git a/Lab-4/catch-falling/game/basket.py b/Lab-4/catch-falling/game/basket.py
index 08e5773..c864c0f 100644
--- a/Lab-4/catch-falling/game/basket.py
+++ b/Lab-4/catch-falling/game/basket.py
@@ -5,4 +5,8 @@ Basket: the player-controlled catcher at the bottom of the screen.
 import pygame

+# How quickly the basket's velocity eases toward the target speed each frame
+# (0..1). High enough to feel responsive, low enough to avoid jerky starts/stops.
+ACCELERATION = 0.35
+

 class Basket:
@@ -13,8 +17,25 @@ class Basket:
         self.height = height
         self.speed = speed
+        self.velocity = 0.0
         self.boosted_frames = 0
         self.normal_speed = speed
         self.boost_speed = speed

+    def move(self, direction, screen_width):
+        """Move one frame. direction is -1 (left), 0 (none) or +1 (right)."""
+        target = direction * self.speed
+        self.velocity += (target - self.velocity) * ACCELERATION
+        if direction == 0 and abs(self.velocity) < 0.1:
+            self.velocity = 0.0
+        self.x += self.velocity
+        self.clamp(screen_width)
+
+    def clamp(self, screen_width):
+        """Keep the whole basket on screen, accounting for its own width."""
+        half = self.width / 2
+        if self.x < half or self.x > screen_width - half:
+            self.x = max(half, min(screen_width - half, self.x))
+            self.velocity = 0.0
+
     def get_rect(self):
         return pygame.Rect(
diff --git a/Lab-4/catch-falling/game/game_engine.py b/Lab-4/catch-falling/game/game_engine.py
index b77810a..05bb120 100644
--- a/Lab-4/catch-falling/game/game_engine.py
+++ b/Lab-4/catch-falling/game/game_engine.py
@@ -37,12 +37,8 @@ class GameEngine:
         if self.game_over:
             return
-        if keys_pressed[pygame.K_LEFT]:
-            self.basket.x -= self.basket.speed
-        if keys_pressed[pygame.K_RIGHT]:
-            self.basket.x += self.basket.speed
```

```
-        # Boundary handling: only clamps against the screen edges, not
-        # accounting for the basket's own width - it can hang half off
-        # either side of the screen.
-        self.basket.x = max(0, min(WIDTH, self.basket.x))
+        # Held keys are read every frame, so movement is continuous; holding
+        # both arrows cancels out instead of favouring one direction.
+        direction = int(bool(keys_pressed[pygame.K_RIGHT])) - int(bool(keys_pressed[pygame.K_LEFT]))
+        self.basket.move(direction, WIDTH)

     def handle_keydown(self, key):
```

Test: holding Right or Left for 400 frames keeps the basket rectangle inside the window, and per-frame steps rise smoothly 2.89 → 3.63 → 4.11 → … → 5 px.

CLAUDE (CLAUDE CODE)

Task 3: Implement controlled object spawning

A new `Spawner` class gives a random 30–75 frame interval and keeps x inside the playable area. Each spawn is at least 90 px from the last three spawn points, and no new object appears while 6 are already on screen.

Iteration: my first version picked x with up to 12 random retries. The test found one pair of consecutive spawns closer than 90 px in 2 minutes of play, because retries can fail and fall back. I changed it to pick uniformly from only the allowed positions, so the rule always holds. (Two other test failures in this step were mistakes in my test script, not the game: it still set the old `frames_until_spawn` field, and it tracked objects by `id()`, which Python reuses.)

```
diff --git a/Lab-4/catch-falling/game/game_engine.py b/Lab-4/catch-falling/game/game_engine.py
index 05bb120..0051e7b 100644
--- a/Lab-4/catch-falling/game/game_engine.py
+++ b/Lab-4/catch-falling/game/game_engine.py
@@ -9,13 +9,11 @@ that Task 1 asks you to fix.
 """

-import random
 import pygame

 from game.basket import Basket
-from game.falling_object import FallingObject
 from game.collision import is_caught
 from game.renderer import WIDTH, HEIGHT
+from game.spawner import Spawner

-SPAWN_INTERVAL_FRAMES = 50
 MAX_MISSES = 5

@@ -25,13 +23,9 @@ class GameEngine:
         self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
         self.objects = []
-        self.frames_until_spawn = 0
+        self.spawner = Spawner(WIDTH)
         self.score = 0
         self.misses = 0
         self.game_over = False

-    def _spawn_object(self):
-        x = random.randint(20, WIDTH - 20)
-        self.objects.append(FallingObject(x=x, y=-14, speed=3))
-
     def handle_input(self, keys_pressed):
         if self.game_over:
@@ -50,8 +44,7 @@ class GameEngine:
             return

-        self.frames_until_spawn -= 1
-        if self.frames_until_spawn <= 0:
-            self._spawn_object()
-            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES
+        new_obj = self.spawner.update(self.objects)
+        if new_obj is not None:
+            self.objects.append(new_obj)

         for obj in self.objects:
diff --git a/Lab-4/catch-falling/game/spawner.py b/Lab-4/catch-falling/game/spawner.py
new file mode 100644
index 0000000..8a95784
--- /dev/null
+++ b/Lab-4/catch-falling/game/spawner.py
@@ -0,0 +1,53 @@
+"""
+Spawner: decides when and where new falling objects appear.
```

```
+
+- Varied timing: the gap between spawns is random within a range.
+- Varied, safe positions: x is always inside the playable area and is kept
+  away from recent spawn points so objects don't stack in one spot.
+- Capped: no new object spawns while the screen already holds the maximum.
+"""
+
+import random
+
+from game.falling_object import FallingObject
+
+MIN_SPAWN_INTERVAL = 30      # frames (0.5 s at 60 FPS)
+MAX_SPAWN_INTERVAL = 75      # frames (1.25 s)
+MAX_OBJECTS_ON_SCREEN = 6
+MIN_SPAWN_GAP = 90           # px between a new spawn and recent spawns
+RECENT_SPAWNS_REMEMBERED = 3
+OBJECT_RADIUS = 14
+
+
+class Spawner:
+    def __init__(self, screen_width, rng=None):
+        self.screen_width = screen_width
+        self.rng = rng or random
+        self.recent_x = []
+        self.frames_until_spawn = 0
+
+    def _pick_x(self):
+        margin = OBJECT_RADIUS + 6
+        lo, hi = margin, self.screen_width - margin
+        positions = range(lo, hi + 1)
+
+        def gap(x):
+            return min((abs(x - r) for r in self.recent_x), default=self.screen_width)
+
+        allowed = [x for x in positions if gap(x) >= MIN_SPAWN_GAP]
+        if allowed:
+            return self.rng.choice(allowed)
+        return max(positions, key=gap)  # no free spot: use the one furthest from recent spawns
+
+    def update(self, objects):
+        """Call once per frame; returns a new FallingObject or None."""
+        self.frames_until_spawn -= 1
+        if self.frames_until_spawn > 0:
+            return None
+        if len(objects) >= MAX_OBJECTS_ON_SCREEN:
+            self.frames_until_spawn = 10  # screen is full: check again shortly
+            return None
+        x = self._pick_x()
+        self.recent_x = (self.recent_x + [x])[-RECENT_SPAWNS_REMEMBERED:]
+        self.frames_until_spawn = self.rng.randint(MIN_SPAWN_INTERVAL, MAX_SPAWN_INTERVAL)
+        return FallingObject(x=x, y=-OBJECT_RADIUS, radius=OBJECT_RADIUS, speed=3)
```

Test: 134 spawns over 2 minutes, x between 20 and 676, intervals from 30 to 75 frames, no consecutive spawns within 90 px; with 6 objects on screen nothing spawns.

CLAUDE (CLAUDE CODE)

Task 4: Add a temporary basket speed boost

SPACE doubles the basket's speed for 3 seconds, then it returns to normal on its own and a 5-second cooldown starts. While active, the basket turns cyan with a glow, and a top-right **BOOST x2** timer bar counts down. Otherwise the corner shows the cooldown or "Boost ready: SPACE".

```
diff --git a/Lab-4/catch-falling/README.md b/Lab-4/catch-falling/README.md
index 481d7b0..2cdc3f9 100644
--- a/Lab-4/catch-falling/README.md
+++ b/Lab-4/catch-falling/README.md
@@ -45,5 +45,5 @@ python main.py
 ```

-**Controls:** Left/Right arrows to move the basket.
+**Controls:** Left/Right arrows to move the basket, SPACE for a 3-second speed boost (5 s cooldown), R to restart after game over.

 ---
diff --git a/Lab-4/catch-falling/game/basket.py b/Lab-4/catch-falling/game/basket.py
index c864c0f..be330e2 100644
--- a/Lab-4/catch-falling/game/basket.py
+++ b/Lab-4/catch-falling/game/basket.py
@@ -9,4 +9,8 @@ import pygame
 ACCELERATION = 0.35

+BOOST_MULTIPLIER = 2.0
```
```

```
+BOOST_DURATION_FRAMES = 180   # 3 s at 60 FPS
+BOOST_COOLDOWN_FRAMES = 300   # 5 s before it can be used again
+

 class Basket:
@@ -19,6 +23,29 @@ class Basket:
         self.velocity = 0.0
         self.boosted_frames = 0
+        self.boost_cooldown = 0
         self.normal_speed = speed
-        self.boost_speed = speed
+        self.boost_speed = speed * BOOST_MULTIPLIER
+
+    @property
+    def is_boosted(self):
+        return self.boosted_frames > 0
+
+    def activate_boost(self):
+        """Start a speed boost if one isn't running and the cooldown is over."""
+        if self.is_boosted or self.boost_cooldown > 0:
+            return False
+        self.boosted_frames = BOOST_DURATION_FRAMES
+        self.speed = self.boost_speed
+        return True
+
+    def update_boost(self):
+        """Count the boost down once per frame; restore normal speed when it ends."""
+        if self.boosted_frames > 0:
+            self.boosted_frames -= 1
+            if self.boosted_frames == 0:
+                self.speed = self.normal_speed
+                self.boost_cooldown = BOOST_COOLDOWN_FRAMES
+        elif self.boost_cooldown > 0:
+            self.boost_cooldown -= 1

     def move(self, direction, screen_width):
diff --git a/Lab-4/catch-falling/game/game_engine.py b/Lab-4/catch-falling/game/game_engine.py
index 0051e7b..15be704 100644
--- a/Lab-4/catch-falling/game/game_engine.py
+++ b/Lab-4/catch-falling/game/game_engine.py
@@ -2,9 +2,8 @@
 GameEngine: owns the basket and all falling objects.

-Starter version: basket movement and spawning both work at a basic
-level (Tasks 2 and 3 ask you to improve them), there's no speed boost
-yet (Task 4 builds it from scratch), and catch detection has two
-known bugs (see game/collision.py and the catch-checking loop below)
-that Task 1 asks you to fix.
+Catches require the object to reach basket height (Task 1), the basket
+stays fully on screen with smooth movement (Task 2), spawning is handled
+by game.spawner.Spawner (Task 3), and SPACE triggers a temporary basket
+speed boost (Task 4).
 """

@@ -39,4 +38,6 @@ class GameEngine:
         if self.game_over and key == pygame.K_r:
             self.__init__()
+        elif not self.game_over and key == pygame.K_SPACE:
+            self.basket.activate_boost()

     def update(self):
@@ -44,4 +45,6 @@ class GameEngine:
             return

+        self.basket.update_boost()
+
         new_obj = self.spawner.update(self.objects)
         if new_obj is not None:
@@ -71,4 +74,5 @@ class GameEngine:
         renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
         renderer.draw_text(surface, font, f"Misses: {self.misses}/{MAX_MISSES}", (10, 36))
+        renderer.draw_boost_status(surface, font, self.basket)

         if self.game_over:
diff --git a/Lab-4/catch-falling/game/renderer.py b/Lab-4/catch-falling/game/renderer.py
index b8add2d..7ebf8f5 100644
--- a/Lab-4/catch-falling/game/renderer.py
+++ b/Lab-4/catch-falling/game/renderer.py
@@ -10,4 +10,7 @@ WINDOW_SIZE = (WIDTH, HEIGHT)
 COLOR_BG = (25, 30, 45)
 COLOR_BASKET = (150, 110, 70)
+COLOR_BASKET_BOOST = (80, 220, 255)
+COLOR_BOOST = (80, 220, 255)
+COLOR_MUTED = (150, 160, 180)
 COLOR_TEXT = (255, 255, 255)

@@ -17,5 +20,10 @@ def draw_scene(surface, basket, objects):
     for obj in objects:
         pygame.draw.circle(surface, obj.color, (int(obj.x), int(obj.y)), obj.radius)
-    pygame.draw.rect(surface, COLOR_BASKET, basket.get_rect(), border_radius=6)
+    rect = basket.get_rect()
+    if basket.is_boosted:
```

```
+        # Glow behind the basket while the boost is active.
+        pygame.draw.rect(surface, (40, 110, 140), rect.inflate(14, 10), border_radius=10)
+    pygame.draw.rect(surface, COLOR_BASKET_BOOST if basket.is_boosted else COLOR_BASKET,
+                     rect, border_radius=6)


@@ -24,4 +32,20 @@ def draw_text(surface, font, text, pos, color=COLOR_TEXT):


+def draw_boost_status(surface, font, basket):
+    """Top-right boost indicator: active timer bar, cooldown, or ready."""
+    from game.basket import BOOST_DURATION_FRAMES
+    x, y, w = surface.get_width() - 230, 10, 220
+    if basket.is_boosted:
+        secs = basket.boosted_frames / 60
+        draw_text(surface, font, f"BOOST x2  {secs:.1f}s", (x, y), COLOR_BOOST)
+        frac = basket.boosted_frames / BOOST_DURATION_FRAMES
+        pygame.draw.rect(surface, (60, 70, 90), (x, y + 28, w, 8), border_radius=4)
+        pygame.draw.rect(surface, COLOR_BOOST, (x, y + 28, int(w * frac), 8), border_radius=4)
+    elif basket.boost_cooldown > 0:
+        draw_text(surface, font, f"Boost in {basket.boost_cooldown / 60:.1f}s", (x, y), COLOR_MUTED)
+    else:
+        draw_text(surface, font, "Boost ready: SPACE", (x, y), COLOR_TEXT)
+
+
 def draw_banner(surface, font, text):
     surf = font.render(text, True, (255, 220, 80))
diff --git a/Lab-4/catch-falling/main.py b/Lab-4/catch-falling/main.py
index a649e0a..986f27e 100644
--- a/Lab-4/catch-falling/main.py
+++ b/Lab-4/catch-falling/main.py
@@ -4,5 +4,6 @@ Catch the Falling Objects (Lab Starter)
 Run with:  python3 main.py

-Controls: Left/Right arrows to move the basket.
+Controls: Left/Right arrows to move the basket, SPACE for a speed boost,
+R to restart after game over.
 """
```

Iteration: frames from the "after" recording showed "Boost ready: SPACE" clipped at the right edge, because pygame fell back to a wider font than I had allowed for. I made the indicator right-align against the window edge, in a follow-up commit.

```
diff --git a/Lab-4/catch-falling/game/renderer.py b/Lab-4/catch-falling/game/renderer.py
index 7ebf8f5..4a21a18 100644
--- a/Lab-4/catch-falling/game/renderer.py
+++ b/Lab-4/catch-falling/game/renderer.py
@@ -35,15 +35,20 @@ def draw_boost_status(surface, font, basket):
     """Top-right boost indicator: active timer bar, cooldown, or ready."""
     from game.basket import BOOST_DURATION_FRAMES
-    x, y, w = surface.get_width() - 230, 10, 220
     if basket.is_boosted:
-        secs = basket.boosted_frames / 60
-        draw_text(surface, font, f"BOOST x2  {secs:.1f}s", (x, y), COLOR_BOOST)
-        frac = basket.boosted_frames / BOOST_DURATION_FRAMES
-        pygame.draw.rect(surface, (60, 70, 90), (x, y + 28, w, 8), border_radius=4)
-        pygame.draw.rect(surface, COLOR_BOOST, (x, y + 28, int(w * frac), 8), border_radius=4)
+        text, color = f"BOOST x2  {basket.boosted_frames / 60:.1f}s", COLOR_BOOST
     elif basket.boost_cooldown > 0:
-        draw_text(surface, font, f"Boost in {basket.boost_cooldown / 60:.1f}s", (x, y), COLOR_MUTED)
+        text, color = f"Boost in {basket.boost_cooldown / 60:.1f}s", COLOR_MUTED
     else:
-        draw_text(surface, font, "Boost ready: SPACE", (x, y), COLOR_TEXT)
+        text, color = "Boost ready: SPACE", COLOR_TEXT
+    label = font.render(text, True, color)
+    # Right-align against the window edge so the text never gets clipped,
+    # whatever font pygame falls back to.
+    x = surface.get_width() - label.get_width() - 10
+    surface.blit(label, (x, 10))
+    if basket.is_boosted:
+        w = label.get_width()
+        frac = basket.boosted_frames / BOOST_DURATION_FRAMES
+        pygame.draw.rect(surface, (60, 70, 90), (x, 38, w, 8), border_radius=4)
+        pygame.draw.rect(surface, COLOR_BOOST, (x, 38, int(w * frac), 8), border_radius=4)
```

Test: speed goes from 5 to 10 on SPACE; pressing again mid-boost doesn't extend it; after 3 s speed is back to 5; SPACE during the cooldown is ignored; after the cooldown it works again.

CLAUDE (CLAUDE CODE)

Final check and deliverables

All tests on the final code:

```
Task 1 tests passed
Task 2 tests passed; first steps: [2.89, 3.63, 4.11, 4.42, 4.62, 4.75]
Task 3 tests passed: 134 spawns, x 20-676, intervals 30-75 frames, max on screen 5,
consecutive spawns closer than 90px: 0
Cap test passed: no spawn while 6 objects are on screen
Task 4 tests passed: boost 5 -> 10 for 3.0 s, then back to 5
```

The "after" video shows: the basket sitting under a high object with no score until it lands; catches at basket height; the boost turning the basket cyan with the timer bar, then the cooldown; and holding Right, where the basket stops fully on screen (centre x = 655 = 700 − 45).

Deliverables in `Lab-4/`: `videos/before.mp4` and `videos/after.mp4` (10 s each), `catch-falling/` (updated code, one commit per task), and this chat history as `Chat_History.pdf`.